



《数据科学导论》—— 文本数据分类



范举 教授

fanj@ruc.edu.cn



提纲



文本分类

- 1. 文本分类问题背景
- 2. 文本的向量化表示
- 3. 文本分类方法：朴素贝叶斯 (Naïve Bayes)
- 4. 文本分析实践



文本分类举例——垃圾信息过滤

From: '' <takworlld@hotmail.com>
Subject: real estate is the only way... gem oalvgkay

Anyone can buy real estate with no money down

Stop paying rent TODAY !

There is no need to spend hundreds or even thousands for similar courses

I am 22 years old and I have already purchased 6 properties using the methods outlined in this truly INCREDIBLE ebook.

Change your life NOW !

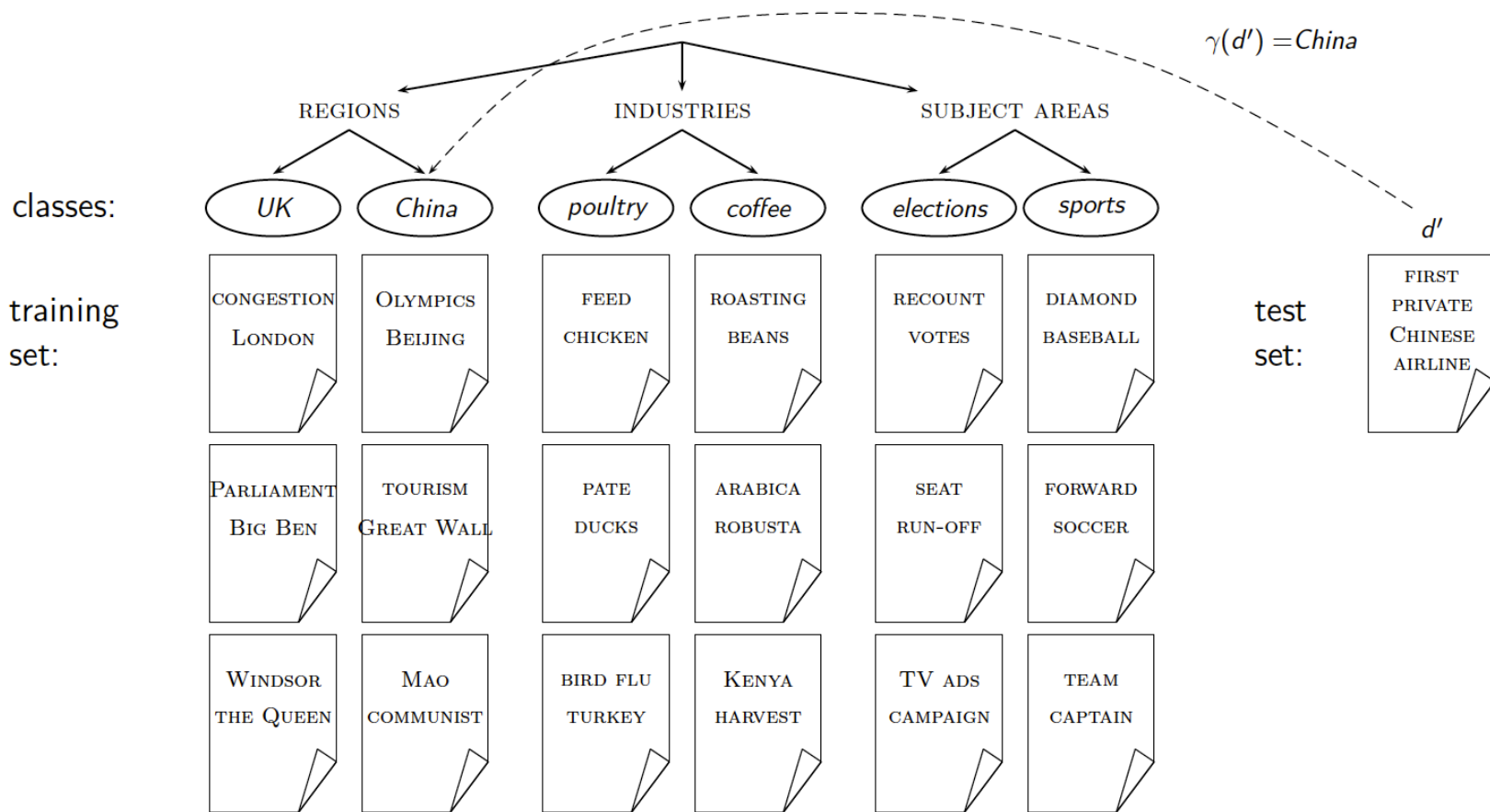
=====
Click Below to order:
<http://www.wholesaledaily.com/sales/nmd.htm>
=====

垃圾Email

标签	短信内容
0	商业秘密的秘密性那是维系其商业价值和垄断地位的前提条件之一
1	南口阿玛施新春第一批限量春装到店啦^ 春暖花开淑女裙、冰蓝色公主衫 气质粉小西装、冰丝女王长半裙
0	带给我们大常州一场壮观的视觉盛宴
0	有原因不明的泌尿系统结石等
0	23年从盐城拉回来的麻麻的嫁妆
1	感谢致电杭州萧山全金釜韩国烧烤店，本店位于金城路xxx号。韩式烧烤等，价格实惠、欢迎惠顾【全金釜韩国烧烤店】
0	这款Uve智能杀菌机器人是扫地机的最佳伴侣
1	一次价值xxx元王牌项目；可充值xxx元店内项目卡一张；可以参与V动好生活百分百抽奖机会一次！预约电话：xxxxxxxxxxx
0	此类皮肤特别容易招惹粉刺、黑头等
1	(长期诚信在本市作各类资格职称（以及印/章、牌、.....等。祥：xxxxxxxxxxxx李伟%

垃圾短信

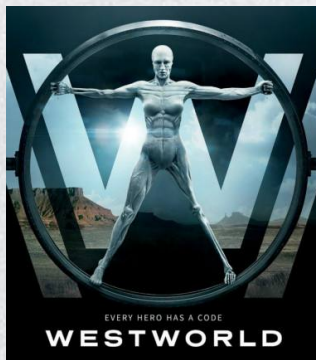
文本分类举例——话题分类



文本分类举例——情感分析

- 情感分析是让计算机拥有人的情感吗？

NO!



文本分类举例——情感分析

- 情感分析是让计算机超越人的情感吗？

NO!



爱，只是一个字而已

但人类千秋和万代

不明白一直到现在

但 A.I.

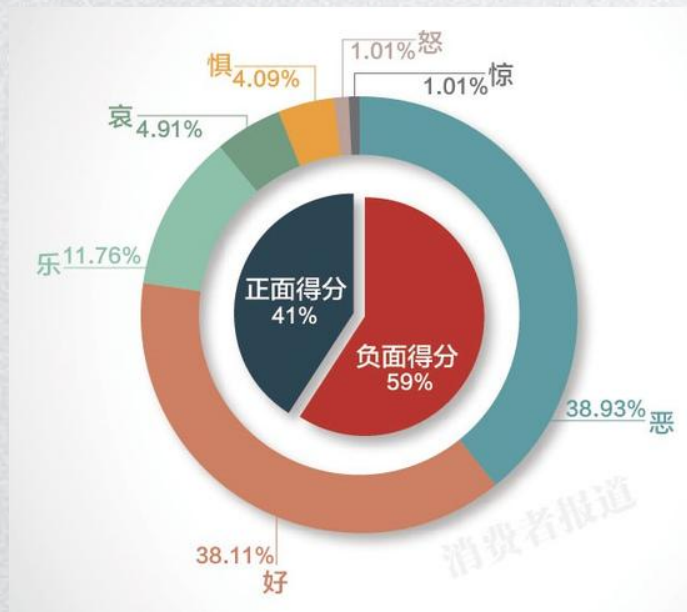
能克服所有问题

——《A.I.爱》

什么是情感分析？

- 情感分析是让计算机.....

识别人的情感



- Sentiment Analysis
- 对带有情感色彩的主观性文本进行分析、处理、归纳和推理，
- 最后得出文本的二元化或者多元化极性分类。



Scherer 情感状态类型

- **情绪 (emotion)** : 有一定原因引发的同步反应
 - 例如悲伤 (sadness)、快乐 (joy)
- **心情 (mood)** : 没有明显原因引发的长期低强度的主观感受变化
 - 例如忧郁 (gloomy)、倦怠 (listless)
- **人际立场 (interpersonal stance)** : 对他人的特定反应
 - 例如疏远 (distant)、冷漠 (cold)
- **态度 (attitude)** : 对特定人或事物的带有主观色彩的偏好或倾向
 - 喜欢 (like)、讨厌 (hate)
- **个性特质 (personal traits)** : 相对稳定的个性倾向和行为趋势
 - 例如焦虑 (nervous)、渴望 (anxious)

案例：商品评论情感分析



HP Officejet 6500A Plus e-All-in-One Color Ink-jet - Fax / copier / printer / scanner

\$89 online, \$100 nearby ★★★★★ 377 reviews

September 2010 - Printer - HP - Inkjet - Office - Copier - Color - Scanner - Fax - 250 sh

Reviews

Summary - Based on 377 reviews



What people are saying

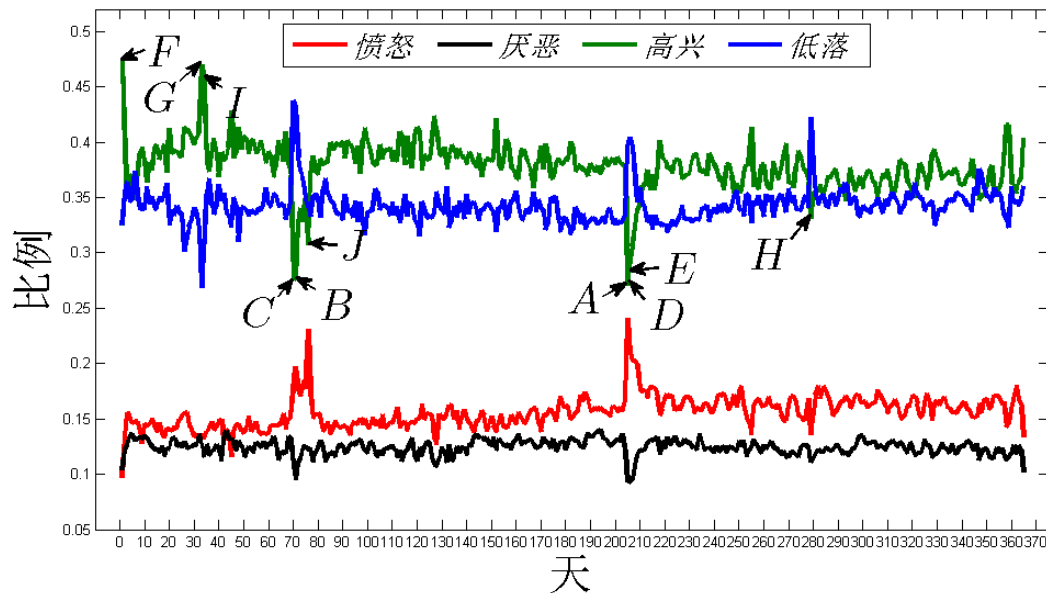
ease of use		"This was very easy to setup to four computers."
value		"Appreciate good quality at a fair price."
setup		"Overall pretty easy setup."
customer service		"I DO like honest tech support people."
size		"Pretty Paper weight."
mode		"Photos were fair on the high quality mode."
colors		"Full color prints came out with great quality."

案例：饭店评论情感分析



评论	打分
口味：不知道是我口高了，还是这家真不怎么样。??我感觉口味确实很一般很一般。上菜相当快，我敢...	2
菜品丰富质量好，服务也不错！很喜欢！	4
说真的，不晓得有人排队的理由，香精香精香精香精，拜拜！	2
菜量实惠，上菜还算比较快，疙瘩汤喝出了秋日的暖意，烧茄子吃出了大阪烧的味道，想吃土豆片也是口...	5
先说我算是娜娜家风荷园开业就一直在这里吃??每次出去回来总想吃一回??有时觉得外面的西式简餐...	4
哎，冲着推荐来的，后悔死了，上菜巨慢不说，服务态度还超级差，东西咸的要死，而且普遍的贵，谁来...	1
超级差评????上菜慢??服务态度超级差、先把汤上来喝饱了??完事菜就上了一个，找服务员催半...	1

案例：微博情感分析



- 北航的先进网络分析研究小组(GANA) 对收集到的2011年的近7000万条微博进行情感分析

案例：微博情感分析

- 如F对应新年，以高兴的情绪为主；
- G，I对应春节，也以高兴的情绪为主；
- A、D、E则对应动车事故，明显看到用户以低落悲伤的情感为主，同时愤怒的情绪比例也明显上升升至全年最高；
- C、B和J对应日本3月份的地震，以低落的情绪为主，但对J点，即2011年03月17日，愤怒的情感比例突然增加，这与当时的盛传碘盐被污染、盐荒等谣言有关；
- H对应苹果前CEO乔布斯逝世，以低落的情绪为主，但同时，愤怒的情绪比例极低，与前面的动车事件和碘盐谣言有明显的区别。

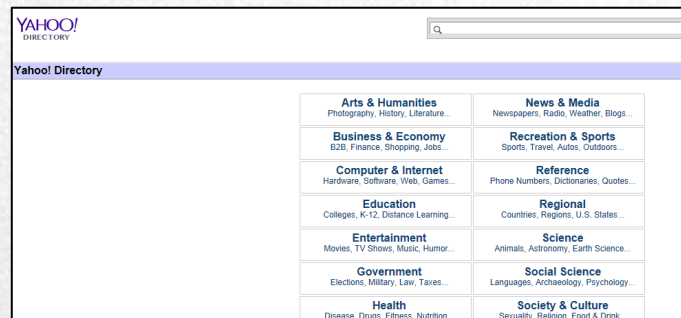
文本分类的形式化定义

- 训练过程：给定
 - 文档空间 X
 - 所有的文档都在此空间进行表示，一般表示为高维向量
 - 文档类别 $C = \{c_1, \dots, c_J\}$
 - 依据目标分类任务人工定义，比如 $C = \{\text{垃圾信息}, \text{正常信息}\}$
 - 带标签的训练集合 $D = \{(d_i, c_i)\}_{i=1}^N, (d_i, c_i) \in X \times C$
 - 基于学习算法，训练得到一个分类器 γ
$$\gamma: X \mapsto C$$
- 应用/测试过程
 - 给定：任意一个（可能在训练集合 D 中未出现）文档 $d \in X$
 - 确定： $\gamma(d) \in C$ ，即对 d 而言最合适的类别

文本分类方法#1：人工分类

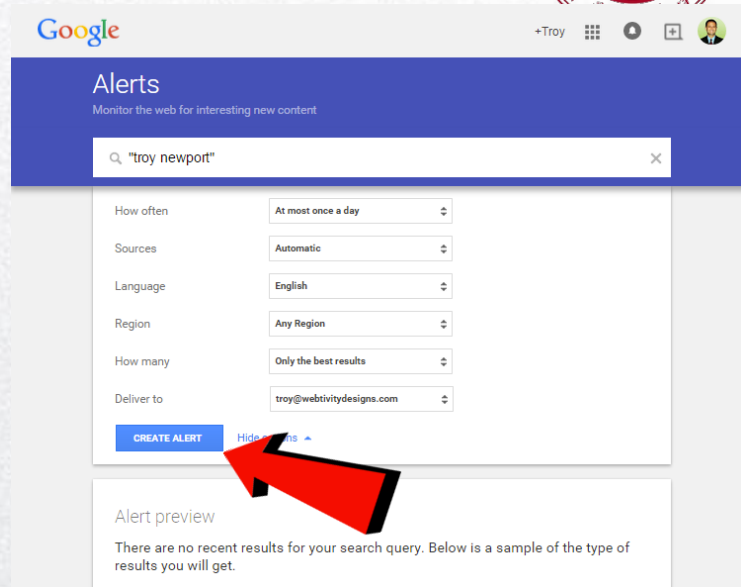
- 雅虎最先提出将网页进行手工分类，ODP、PubMed也进行了类似的实践
 - 准确率高（如果雇佣专家进行分类）
 - 稳定（如果问题和团队规模可控）
- 规模化难题
 - 费用太高
 - 速度慢
 - 专家团队人数太多后，准确率难以保证

需要自动化的分类方法！



文本分类方法#2：基于规则的分类

- 例如：Google alerts是基于规则的分类
 - 通过辅助工具创建分类规则
 - 通常规则为布尔表达式
如 “标题中含有 homepage”
AND “URL中含有~”
- 优点：准确率高
 - 如果雇佣专家进行规则的编写和维护
 - 易于理解
- 缺点：规则维护任务繁重
 - 分类规则甚至类别经常发生变化
 - 规则冲突问题



Google Alerts

Monitor the web for interesting new content

Q "troy newport"

How often: At most once a day

Sources: Automatic

Language: English

Region: Any Region

How many: Only the best results

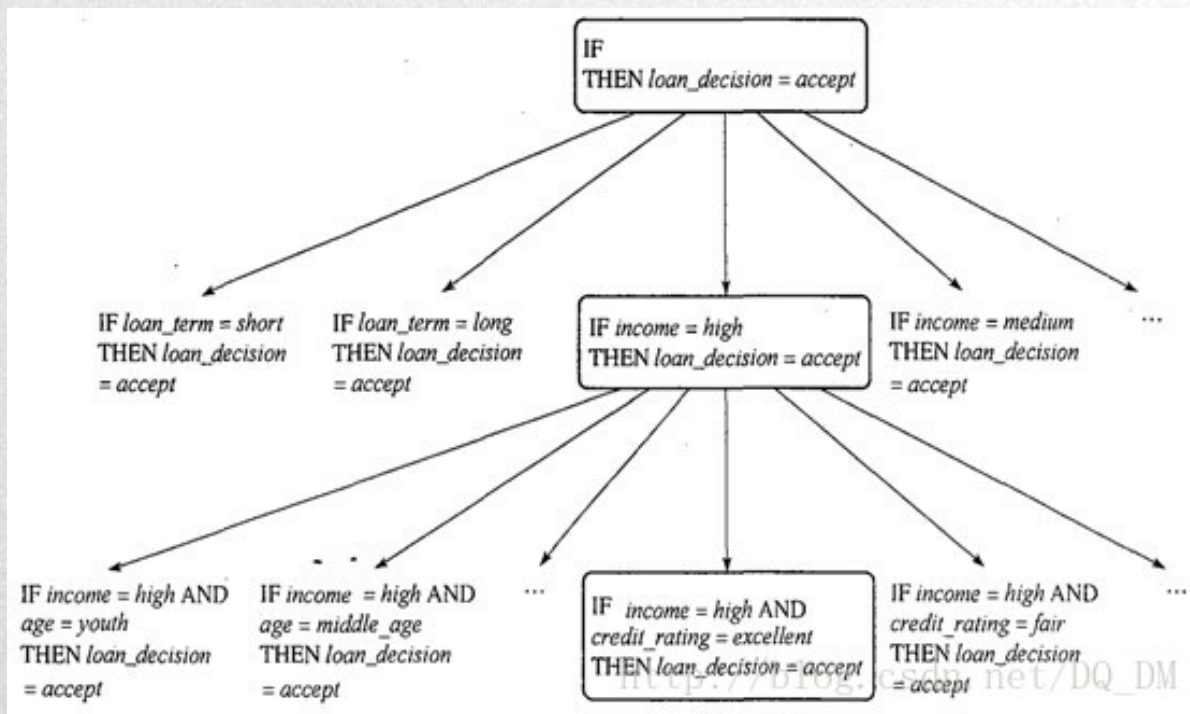
Deliver to: troy@webtivitydesigns.com

CREATE ALERT

Alert preview

There are no recent results for your search query. Below is a sample of the type of results you will get.

规则分类举例





文本分类方法#3：基于统计学习的分类

- 把文本分类问题看成一个机器学习问题
 - 基于**监督学习算法**，训练得到一个分类器 $\gamma: X \mapsto C$
 - 把 γ 应用于新文档 d ： $\gamma(d) \in C$
- 没有免费的午餐
 - 需要手工标注的训练数据（数据驱动）
 - 需要把文本表达为合适的特征向量



提纲



文本分类

- 1. 文本分类问题背景
- 2. 文本的向量化表示
- 3. 文本分类方法：朴素贝叶斯 (Naïve Bayes)
- 4. 文本分析实践

文本的向量表示

- 文本的向量表示是将每一个文本表示为向量空间的一个向量
- 基于文本的向量表示，可以进行多种文本分析任务
 - 文本相似度计算
 - 文本分类
 - 文本聚类
 -



文本的向量表示

• 例：文本相似度计算

专栏：陕甘宁南线进行前哨战蒋军开始犯我关中亲蒋报纸供认“停战令”为废纸【新华社延安二十日电】陕甘宁边区南线蒋军开始前哨侦察战，蒋军胡宗南部二十九军（原三十七集团军）三十六师一六五旅三六七团一营，于十六日晚八时向关中分区淳（化）耀（县）柳林镇之文王山开始攻击，激战三小时；复企图向武王山攻击八路军的侧背，至发电时仍对峙中。又十九日中部附近蒋军一六五旅四九五团第三营一部向边区八路军阵地进攻。【新华社延安二十日电】蒋介石进犯陕甘宁边区与延安的军事行动与其扩大内战的部署已无法掩盖，甚至日前尚在否认蒋军对延安军事行动之各报亦不得开始报导这一事实。据合众社十八、十九两日南京电报导，蒋介石在秘密的“新收复区绥靖会议”上说：“政府能够在五个月之内消灭共产党的军队。”会议自十七日起举行三日，此次会议出席政府要员军方及军方首领和各省主席五十余人，在开幕时国防部长白崇禧继蒋介石向大会致词，主张采取蒋介石江西“剿共”的老政策：“三分军事，七分政治。”合众社称，一切征象均证明政府以进攻中共中枢城市延安为起点的广泛攻势已极为紧张。上述蒋介石的声明，即为主要征象之一。另一征象为十九日蒋已命令现在南京参加“国大”的重要军事首领返回前线，最主要的征象之一为过去一周内亲政府的报纸强调蒋介石“停战令”只是一纸废纸，而尽力把责任推给所谓“共军在各线发动攻势”。

[0.07631530790990465, 0.06431599860099027, 0.06700221901780429, 0.056434730860138435, 0.013095526224925711, 0.05726946622899936, 0.07730411904395502, 0.04061600367161102, 0.056778326434723185, 0.0747835391137996, 0.03565911164449718, 0.07190902691127214, 0.05364890696524792, 0.13377162714757948, 0.062413476817320374, 0.05343970880686189, 0.11791273379275316, 0.16484543807961138, 0.17276249224942222, 0.055848075604964895]

从13739篇文章中检索

检索结果

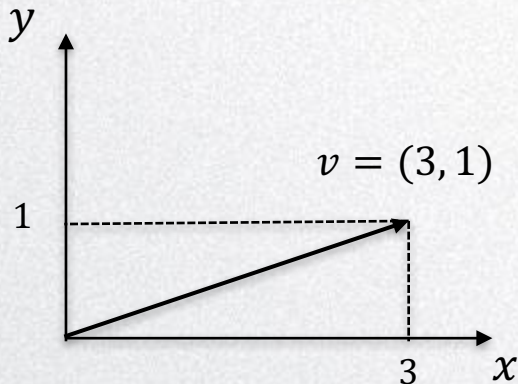
相似度 = 0.4630335200941314

专栏：蒋军刘汝珍米文和部新兵占百分之八十大都是被抓来的农民小贩【本报冀鲁豫前线一日电】在定陶西台集战斗中放下武器之蒋军刘汝珍米文和部士兵多系新兵，据他们自称：该两部蒋军中一个月至三个月军龄之士兵占百分之八十以上，甚至除去班长外全班都是新兵，这些新兵多系抓来的农民和少数小商贩，他们反对内战不甘心替蒋介石卖命。在这一次开来前线时，中途逃亡很多，只二二〇团的两个营就逃亡了二百多名，没有逃脱的就在战场上枪未发成群的把枪交给八路军。西台集被我解放之蒋军新兵纷纷控诉蒋军罪行，他们说：“只有把我们的家解放了，才不再受蒋介石的欺压。”为达到这个目的，他们很多已自动参加了八路军。

[0.07631530790990465, 0.06431599860099027, 0.06700221901780429, 0.056434730860138435, 0.013095526224925711, 0.05726946622899936, 0.07730411904395502, 0.04061600367161102, 0.056778326434723185, 0.0747835391137996, 0.03565911164449718, 0.07190902691127214, 0.05364890696524792, 0.13377162714757948, 0.062413476817320374, 0.05343970880686189, 0.11791273379275316, 0.16484543807961138, 0.17276249224942222, 0.055848075604964895]

向量

- **向量** (vector) , 是具有大小和方向的量。
- 与向量对应的量叫做标量, 标量只有大小, 没有方向。
- 向量的几何表示
 - 可以形象化地表示为带箭头的线段。
 - 箭头所指: 代表向量的方向;
 - 线段长度: 代表向量的大小。
- 向量代数表示
 - 用黑体英文字母表示 $\mathbf{v}$
 - 或者书写时在字母顶上加一小箭头 $\vec{v}$
- 向量的坐标表示
 - 在坐标系中使用一组有序实数表示向量



向量

- 向量的长度
 - 向量 $\boldsymbol{v}$ 的长度记为 $|\boldsymbol{v}|$
 - 向量的长度是一个非负实数，即
- 向量的维度
 - 向量维度是向量分量的个数
 - 例如 $\boldsymbol{v} = (x, y)$ 是二维向量， $\boldsymbol{v} = (x, y, z)$ 是三维向量
- 向量长度的计算

$$\boldsymbol{v} = (x, y), \text{ 则 } |\boldsymbol{v}| = \sqrt{x^2 + y^2}$$

- 向量的内积

$$\boldsymbol{v}_1 = (x_1, y_1), \boldsymbol{v}_2 = (x_2, y_2), \text{ 则 } \boldsymbol{v}_1 \cdot \boldsymbol{v}_2 = x_1 x_2 + y_1 y_2$$



向量



- 练习：
 - $v_1 = (3, -4)$, $v_2 = (5, 12)$
 - 求 v_1 和 v_2 的长度
 - 求 v_1 和 v_2 的内积



向量

- 例：

- $v_1 = (3, -4), v_2 = (5, 12)$
- 求 v_1 和 v_2 的长度

$$|v_1| = \sqrt{3^2 + (-4)^2} = 5$$

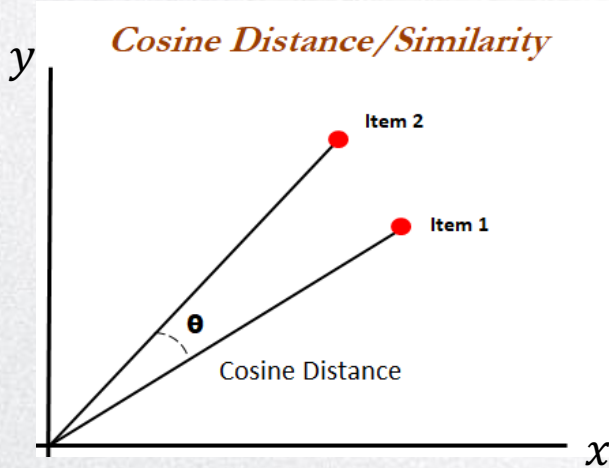
$$|v_2| = \sqrt{5^2 + 12^2} = 13$$

- 求 v_1 和 v_2 的内积

$$v_1 \cdot v_2 = 3 \times 5 + (-4) \times 12 = -33$$

向量

- 向量的余弦相似度
 - 通过测量两个**向量**的**夹角**的**余弦值**来度量它们之间的相似度
 - 余弦相似度范围为 $[-1, 1]$
 - 余弦相似度越大，夹角越小，向量越相似



- 余弦相似度计算

$$\text{similarity}(\mathbf{a}, \mathbf{b}) = \cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|}$$

高维向量计算

- 高维向量计算和二维向量计算类似

- 向量长度

$$\mathbf{x} = (x_1, x_2, \dots, x_n), \text{ 则 } |\mathbf{x}| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

- 向量的内积

$$\mathbf{x} = (x_1, x_2, \dots, x_n), \mathbf{y} = (y_1, y_2, \dots, y_n)$$

$$\text{则 } \mathbf{x} \cdot \mathbf{y} = x_1 y_1 + x_2 y_2 + \dots + x_n y_n$$

- 余弦相似度

$$\mathbf{x} = (x_1, x_2, \dots, x_n), \mathbf{y} = (y_1, y_2, \dots, y_n)$$

$$\text{则 similarity}(\mathbf{x}, \mathbf{y}) = \cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{|\mathbf{x}| |\mathbf{y}|}$$

文本词袋模型 Word Bags

- 基本目标：通过单词集合来表示一篇文档

ID	评论
0	饭店 味道 好 好 好
1	饭店 味道 差
2	饭店 位置 好



词典 Vocabulary
{饭店, 味道, 位置, 好, 差}



ID	表示
0	(1, 1, 0, 1, 0)
1	(1, 1, 0, 0, 1)
2	(1, 0, 1, 1, 0)

0 - 相应位置的词没出现
1 - 相应位置的词出现



TF-IDF向量

- TF-IDF是一种统计方法，用以评估一个词在一篇文章和整个语料库的重要程度
- 假设词表大小为 N ，则每篇文章可以表示为一个维度为 N 的向量

$$\mathbf{x} = (x_1, x_2, \dots, x_N)$$

其中 x_i 为词 w_i 的TF-IDF值, $i = 1, 2, \dots, N$ 。 $\mathbf{x}$ 是文章的向量表示。

- 基于文章的向量表示 $\mathbf{x}$ ，可以进行文本的相似度计算

	单词1	单词2	单词3	单词4	单词5	...	单词N
文章1	✓	✓	✓	✓	✓	...	✓
文章2	✓	✓	✓	✓	✓	...	✓
文章3	✓	✓	✓	✓	✓	...	✓

← 文章2的
向量表示



TF-IDF模型

DF: Document Frequency

- 如何度量单词对文档的重要性
 - 一篇文档中重复出现的单词很重要
 - 大多数文档中都出现的单词不重要

• TF: term frequency

– $tf(t, d) = \frac{Frequency(t, d)}{|d|}$

term frequency

- $Frequency(t, d)$: 单词 t 在文档 d 中出现了几次
- $|d|$: 文档 d 总共出现了多少个单词 (包含重复)

• IDF: inverse document frequency

– $idf(t) = \log \frac{|D|}{1+df(t, f)}$

$\log_{10} \frac{|D|}{1+df}$

- $df(t, f)$: 文档库 D 中有多少篇文档包含单词 t
- $|D|$: 文档库中总共有多少篇文档

TF-IDF模型

- TF: term frequency

- $tf(t, d) = \frac{Frequency(t, d)}{|d|}$

- $Frequency(t, d)$: 单词 t 在文档 d 中出现了几次
 - $|d|$: 文档 d 总共出现了多少个单词（包含重复）

- IDF: inverse document frequency

- $idf(t) = \log \frac{|D|}{1+df(t, f)}$

- $df(t, f)$: 文档库 D 中有多少篇**文档**包含单词 t
 - $|D|$: 文档库中总共有多少篇文档

- TF-IDF模型 : $tf(t, d) \times idf(t)$

练习题

- 计算下面例子中的TF-IDF分值，并计算文档0和1的相似性

ID	评论
0	饭店 味道 好 好 好
1	饭店 味道 差
2	饭店 位置 好



词典 Vocabulary
 {饭店, 味道, 位置, 好, 差}



ID	TF-IDF表示
0	$\frac{1}{3} \times \log \frac{3}{1}, \frac{1}{3} \times \log 1, 0, \frac{3}{5} \times \log 1, 0$
1	$\frac{1}{3} \times \log \frac{3}{1}, \frac{1}{3} \times \log 1, 0, 0, \frac{1}{3} \times \log \frac{3}{1}$
2	$\frac{1}{3} \times \log \frac{3}{1}, 0, \frac{1}{3} \log \frac{3}{2}, \frac{1}{3} \times \log 1, 0$

$$\log_{10} \frac{|D|}{1 + df(x)} = \log \frac{3}{1+df}$$



提纲



文本分类

- 1. 文本分类问题背景
- 2. 文本的向量化表示
- 3. 文本分类方法：朴素贝叶斯 (Naïve Bayes)
- 4. 文本分析实践

复习：什么是贝叶斯

- 一个例子：红蓝石子游戏 (Marble Game)

- 你面前有一个罐子，里面放三颗石子

- 罐子的可能的类型有两种

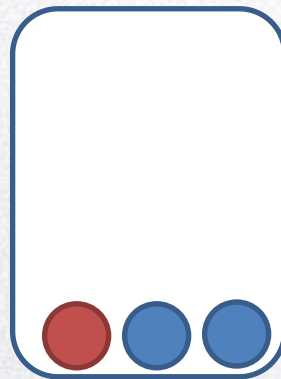
- 蓝罐子：2颗蓝色、1颗红色

- 红罐子：2颗红色、1颗蓝色

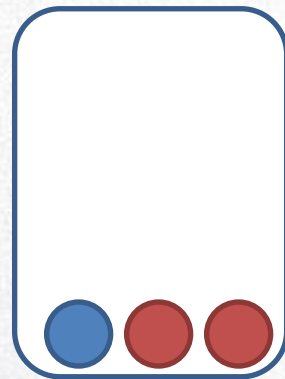
- 罐子类型未知但等概率，你通过摸石子判断

- 如果摸出一颗红色石子，判断是蓝罐子的概率是多少？

蓝罐子



红罐子



A: 1/3 B: 1/2

C: 0 D: 2/3

摸出红: $\frac{1}{2} \times \frac{1}{3} + \frac{2}{3} \times \frac{1}{2} = \frac{1}{2}$

$$P(B|A) = \frac{\frac{1}{2} \times \frac{1}{3}}{\frac{1}{2}} = \frac{1}{3}$$

贝叶斯公式

- 定义随机变量
 - 随机变量 C ：表示罐子的类型
 - $C = c_1$ ：表示罐子属于蓝色为主型
 - $C = c_2$ ：表示罐子属于红色为主型
 - 随机变量 X ：表示摸出石子的颜色
 - $X = r$ ：表示摸出了红色石子
 - $X = b$ ：表示摸出了蓝色石子
- 需要计算的概率

$$P(C = c_1|X = r) = \frac{P(X = r|C = c_1)P(C = c_1)}{P(X = r|C = c_1)P(C = c_1) + P(X = r|C = c_2)P(C = c_2)}$$

复习：什么是贝叶斯

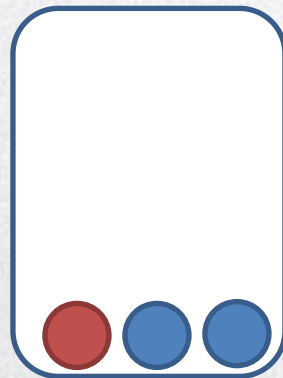
- 一个例子：红蓝石子游戏 (Marble Game)
 - 罐子类型未知但等概率，你通过摸石子判断
 - 问题2：如果一组同学一个接一个的摸石子，每个同学公布颜色后，再把石子放回

- 第一个同学摸出蓝色，蓝罐子概率？
- 第二个同学摸出红色，蓝罐子概率？
- 第三个同学摸出蓝色，蓝罐子概率？

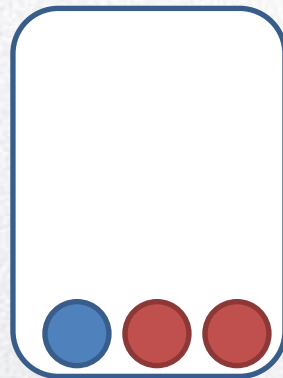


A: $1/3$ B: $1/2$ C: 0 D: $2/3$

蓝罐子



红罐子



朴素贝叶斯 (Naïve Bayes)

- 朴素贝叶斯属于概率分类器，给定一个文档 d ，其属于类别 c 的概率为

$$P(c|d) \propto P(c) \prod_{1 \leq k \leq n_d} P(t_k|c)$$

- n_d ：文档长度
- $P(t_k|c)$ ：给定类别 c ，出现单词 t_k 的概率
 - 通过单词 t_k 来度量类别 c 是否是文档 d 的正确类别
- $P(c)$ ：类别 c 的先验概率
 - 如果一个文档没有任何的迹象（单词）表明它属于类别 c 的概率要大于属于 c' 的概率，那么则选择先验概率较大的类别

$$P(c|d) = \frac{P(c) \cdot P(d|c)}{P(d)}$$

$$\prod_{1 \leq k \leq n_d} P(t_k|c)$$

最大化后验概率

- 朴素贝叶斯的目标
 - 为文档找到“最好”的类
- 什么是最好的类？
 - 能够最大化后验概率的类别 (maximum a posteriori (MAP) class), 记为 c_{map}

$$c_{\text{map}} = \arg \max_{c \in \mathcal{C}} \hat{P}(c|d) = \arg \max_{c \in \mathcal{C}} \hat{P}(c) \prod_{1 \leq k \leq n_d} \hat{P}(t_k|c)$$

通过log把乘法变成加法

- 计算机表达概率相乘所遇到的问题
 - 如果把很多个概率（0~1之间的数字）相乘，会很快得到低于计算机所能表达精度的数字（floating point underflow），从而得到0概率
- 解决方案： $\log(xy) = \log(x) + \log(y)$
 - 通过求log和的方式表达积的log
- 注意： $\log$ 是一个单增函数，取log后的函数并不改变其最大值点
- 因此，Naïve Bayes实际去计算

$$c_{\text{map}} = \arg \max_{c \in \mathcal{C}} \left[\log \hat{P}(c) + \sum_{1 \leq k \leq n_d} \log \hat{P}(t_k | c) \right]$$

朴素贝叶斯分类器

c 的先验概率，即类别 c 在总体数据中出现的频率

单词 t_k 对类别 c 的权重，表明当出现此单词时表明其类别应该为 c 的信号强度

$$c_{\text{map}} = \arg \max_{c \in \mathcal{C}} \left[\log \hat{P}(c) + \sum_{1 \leq k \leq n_d} \log \hat{P}(t_k | c) \right]$$

选取概率最大的类别

Log先验概率与词权重的和：总体而言文档属于此类别的概率

参数估计

- 给定训练数据之后，如何估计上述概率（参数）？
- 需要估计的参数包括
 - 先验概率： $P(c)$, for all $c \in \mathcal{C} = \{c_1, \dots, c_J\}$
 - 条件概率： $P(t_k|c)$, for all $t_k \in V$, and $c \in \mathcal{C} = \{c_1, \dots, c_J\}$, 其中 V 为所有可能的单词

$$c_{\text{map}} = \arg \max_{c \in \mathcal{C}} \left[\log \hat{P}(c) + \sum_{1 \leq k \leq n_d} \log \hat{P}(t_k|c) \right]$$

参数估计方法：最大似然估计

- 先验概率： $\hat{P}(c) = \frac{N_c}{N}$

属于c的文档数

– N_c : 训练集中属于类别c的文档数目； N : 训练集中所有文档数目

- 条件概率： $\hat{P}(t|c) = \frac{T_{ct}}{\sum_{t' \in V} T_{ct'}}$

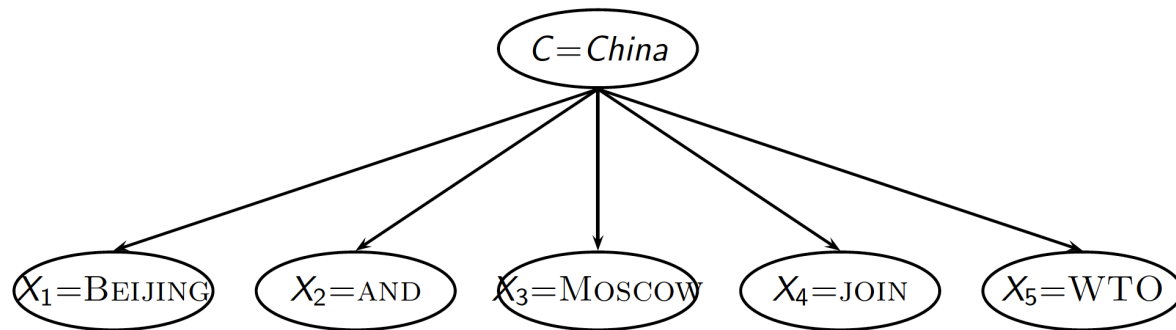
t在c出现的次数

总词在c总次数

– T_{ct} : 在训练集中，属于类别c的文档中包含单词t的次数（包括重复出现）

Naïve Bayes独立性假设：单词t在类别c中出现的概率其所出现的位置和上下文无关

最大似然估计的问题：零概率



$$P(\text{China}|d) \propto P(\text{China}) \cdot P(\text{BEIJING}|\text{China}) \cdot P(\text{AND}|\text{China}) \\ \cdot P(\text{MOSCOW}|\text{China}) \cdot P(\text{JOIN}|\text{China}) \cdot P(\text{WTO}|\text{China})$$

- 如果单词WTO从来没有在类别China中出现过

$$\hat{P}(\text{WTO}|\text{China}) = \frac{T_{\text{China},\text{WTO}}}{\sum_{t' \in V} T_{\text{China},t'}} = \frac{0}{\sum_{t' \in V} T_{\text{China},t'}} = 0$$

最大似然估计的问题：零概率

- 在训练数据中，属于China的文档中从来没有出现过单词WTO，则

$$\hat{P}(WTO|China) = \frac{T_{China,WTO}}{\sum_{t' \in V} T_{China,t'}} = \frac{0}{\sum_{t' \in V} T_{China,t'}} = 0$$

- 依据分类规则，则所有含有WTO的文档属于China类别的概率为0：
 - $P(China|d) = 0$ ，如果 d 中含有单词WTO

$$c_{\text{map}} = \arg \max_{c \in \mathcal{C}} \hat{P}(c|d) = \arg \max_{c \in \mathcal{C}} \hat{P}(c) \prod_{1 \leq k \leq n_d} \hat{P}(t_k|c)$$

如何避免零概率：概率平滑

- 最大似然条件概率估计：

$$\hat{P}(t|c) = \frac{T_{ct}}{\sum_{t' \in V} T_{ct'}}$$

- 平滑化：在所有的统计次数上加一次

$$\hat{P}(t|c) = \frac{T_{ct} + 1}{\sum_{t' \in V} (T_{ct'} + 1)} = \frac{T_{ct} + 1}{\sum_{t' \in V} T_{ct'} + B}$$

- $B=|V|$ ：字典中所含有不同的单词数目

朴素贝叶斯小结

- 训练：平滑化基于训练数据的概率数值，避免零概率
- 应用/测试：给定一个文档，
 - 计算每一个类别的log先验概率
 - 计算每一个文档中所出现的单词的log后验概率
 - 求和
 - 将分值最高的类别作为预测的类

朴素贝叶斯训练算法

TRAINMULTINOMIALNB($\mathbb{C}, \mathbb{D}$)

```
1   $V \leftarrow \text{EXTRACTVOCABULARY}(\mathbb{D})$ 
2   $N \leftarrow \text{COUNTDOCS}(\mathbb{D})$ 
3  for each  $c \in \mathbb{C}$ 
4  do  $N_c \leftarrow \text{COUNTDOCSINCLASS}(\mathbb{D}, c)$ 
5      $\text{prior}[c] \leftarrow N_c / N$ 
6      $\text{text}_c \leftarrow \text{CONCATENATETEXTOFALLDOCSINCLASS}(\mathbb{D}, c)$ 
7     for each  $t \in V$ 
8     do  $T_{ct} \leftarrow \text{COUNTTOKENSOFTERM}(\text{text}_c, t)$ 
9     for each  $t \in V$ 
10    do  $\text{condprob}[t][c] \leftarrow \frac{T_{ct}+1}{\sum_{t'} (T_{ct'}+1)}$ 
11 return  $V, \text{prior}, \text{condprob}$ 
```

朴素贝叶斯应用/测试

```
APPLYMULTINOMIALNB( $\mathbb{C}$ ,  $V$ ,  $prior$ ,  $condprob$ ,  $d$ )  
1   $W \leftarrow \text{EXTRACTTOKENSFROMDOC}(V, d)$   
2  for each  $c \in \mathbb{C}$   
3  do  $score[c] \leftarrow \log prior[c]$   
4    for each  $t \in W$   
5    do  $score[c] + = \log condprob[t][c]$   
6  return  $\arg \max_{c \in \mathbb{C}} score[c]$ 
```

练习

	docID	words in document	in $c = \text{China}$?
training set	1	Chinese Beijing Chinese	yes
	2	Chinese Chinese Shanghai	yes
	3	Chinese Macao	yes
	4	Tokyo Japan Chinese	no
test set	5	Chinese Chinese Chinese Tokyo Japan	?

$$\hat{P}(c) = \frac{N_c}{N} = \frac{3}{4}$$

$$\hat{P}(t|c) = \frac{T_{ct} + 1}{\sum_{t' \in V} (T_{ct'} + 1)} = \frac{T_{ct} + 1}{(\sum_{t' \in V} T_{ct'}) + B}$$

$$c_{\text{map}} = \arg \max_{c \in \mathcal{C}} \hat{P}(c|d) = \arg \max_{c \in \mathcal{C}} \hat{P}(c) \prod_{1 \leq k \leq n_d} \hat{P}(t_k|c)$$

训练过程 - 1

- 步骤1：提取词典 (Vocabulary)
 - $V = \{\text{Chinese, Beijing, Shanghai, Macao, Tokyo, Japan}\}$
- 步骤2：计算训练文档个数
 - $N = 4$
- 步骤3：针对正例文档，表示为 c
 - 计算正例文档个数 $N_c = 3$
 - 计算先验概率 $P(c) = \frac{3}{4} = 0.75$
 - 将正例文档进行合并形成文本 text_c

训练过程 - 2

- 步骤3：针对正例文档，即 $c=China$
 - 针对词典 V 中的每个单词，进行如下计算

$$P(Chinese|c) = \frac{5+1}{8+6} = \frac{3}{7}$$

$$P(Beijing|c) = \frac{1+1}{8+6} = \frac{1}{7}$$

$$P(Shanghai|c) = \frac{1+1}{8+6} = \frac{1}{7}$$

$$P(Macao|c) = \frac{1+1}{8+6} = \frac{1}{7}$$

$$P(Tokyo|c) = \frac{0+1}{8+6} = \frac{1}{14}$$

$$P(Japan|c) = \frac{0+1}{8+6} = \frac{1}{14}$$

$$P(t|c) = \frac{T_{ct} + 1}{\sum_{t' \in V} T_{ct'} + |B|}$$

$$\frac{P(Chinese|c)}{P(Beijing|c)} = \frac{\frac{3+1}{8+6}}{\frac{1+1}{8+6}} = \frac{6}{14}$$

训练过程 - 3

- 步骤4：针对负例文档，表示为 $\bar{c}$
 - 计算先验概率 $P(\bar{c}) = \frac{1}{4} = 0.25$
 - 将负例文档进行合并形成文本 $\text{text}_{\bar{c}}$
 - $P(\text{Chinese}|\bar{c}) = \frac{1+1}{3+6} = \frac{2}{9}$
 - $P(\text{Beijing}|\bar{c}) = \frac{0+1}{3+6} = \frac{1}{9}$
 - $P(\text{Shanghai}|\bar{c}) = \frac{0+1}{3+6} = \frac{1}{9}$
 - $P(\text{Macao}|\bar{c}) = \frac{0+1}{3+6} = \frac{1}{9}$
 - $P(\text{Tokyo}|\bar{c}) = \frac{1+1}{3+6} = \frac{2}{9}$
 - $P(\text{Japan}|\bar{c}) = \frac{1+1}{3+6} = \frac{2}{9}$

测试（预测）过程

- 给定测试文本
 - Chinese Chinese Chinese Tokyo Japan
- 计算正例的后验概率
 - $P(c|d) = \frac{3}{4} \cdot \left(\frac{3}{7}\right)^3 \cdot \frac{1}{14} \cdot \frac{1}{14} \approx 0.0003$
- 计算负例的后验概率
 - $P(\bar{c}|d) = \frac{1}{4} \cdot \left(\frac{2}{9}\right)^3 \cdot \frac{2}{9} \cdot \frac{2}{9} \approx 0.0001$
- 由于 $P(c|d) > P(\bar{c}|d)$, 预测测试文本的类别为正例
- 请你计算以下文本
 - Beijing Tokyo Japan

$\log V + \log L$

朴素贝叶斯时间复杂度

mode	time complexity
training	$\Theta(\mathbb{D} L_{ave} + \mathbb{C} V)$
testing	$\Theta(L_a + \mathbb{C} M_a) = \Theta(\mathbb{C} M_a)$

- L_{ave} : 平均文档长度 ; L_a : 测试文档长度 ; M_a : 测试文档中不同的单词数 ; $\mathbb{D}$: 训练文档集合 ; V : 字典集合 ; $\mathbb{C}$: 类别集合
- $\Theta(|\mathbb{D}|L_{ave})$: 训练文档中所有的统计计数
- $\Theta(|\mathbb{C}||V|)$: 基于统计计数计算参数值
- 一般而言 : $|\mathbb{C}||V| < |\mathbb{D}|L_{ave}$
- 测试时间复杂度 : 对文档长度而言为线性
- 结论 : 朴素贝叶斯在训练和测试中均为线性时间复杂度 (对文档长度而言) , 最优



如何理解朴素贝叶斯？

贝叶斯公式

- 分类目标：找出文档最可能属于的类别

$$c_{\text{map}} = \arg \max_{c \in \mathcal{C}} P(c|d)$$

- 应用贝叶斯公式 $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$

$$c_{\text{map}} = \arg \max_{c \in \mathcal{C}} \frac{P(d|c)P(c)}{P(d)}$$

- 由于 $P(d)$ 与类别选取没有关系，因此可以去除

$$c_{\text{map}} = \arg \max_{c \in \mathcal{C}} P(d|c)P(c)$$

庞大的条件概率参数空间

$$\begin{aligned}c_{\text{map}} &= \arg \max_{c \in \mathcal{C}} P(d|c)P(c) \\ &= \arg \max_{c \in \mathcal{C}} P(\langle t_1, \dots, t_k, \dots, t_{n_d} \rangle | c)P(c)\end{aligned}$$

- 逐个求解 $P(\langle t_1, \dots, t_k, \dots, t_{n_d} \rangle | c)P(c)$ 非常困难
 - 数目过于庞大： $|V|^{n_d}$ ， $t_k \in V$ 可以为任意一个单词
 - 需要海量的训练数据去估计所有的 $P(\langle t_1, \dots, t_k, \dots, t_{n_d} \rangle | c)$
- 数据稀疏问题

假设一：条件独立假设

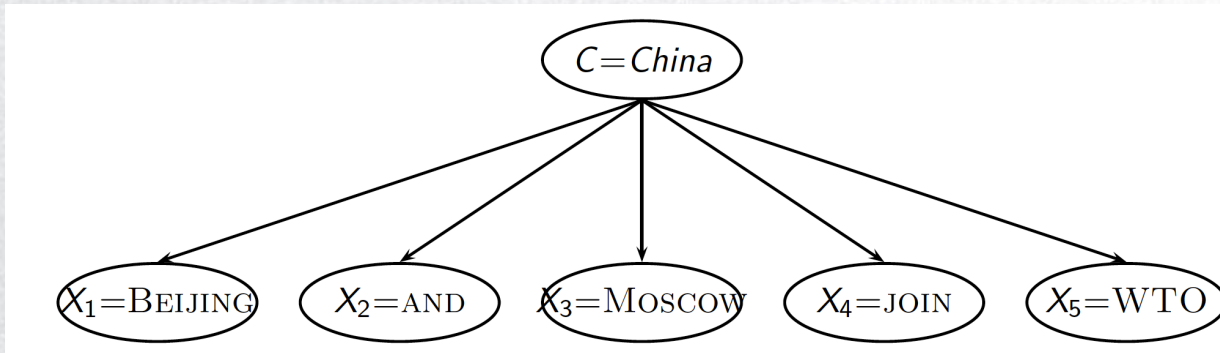
- 条件独立假设：在给定类别的条件下，文档中的单词出现概率独立

$$P(d|c) = P(\langle t_1, \dots, t_k, \dots, t_{n_d} \rangle | c) = \prod_{1 \leq k \leq n_d} P(X_k = t_k | c)$$

- 独立： $P(AB) = P(A)P(B)$
- 给定类别 c ，文档 d 中所有的单词出现的联合概率 $P(t_1, \dots, t_{n_d} | c)$ 可以分解为各个单词出现概率 $P(X_k = t_k | c)$ 的乘积，而 $P(X_k = t_k | c)$ 可以表示为

$$\hat{P}(t|c) = \frac{T_{ct} + 1}{(\sum_{t' \in V} T_{ct'}) + B}$$

生成模型



- $P(c|d) \propto P(c) \prod_{1 \leq k \leq n_d} P(t_k|c)$
 - 依据 $P(c)$ 生成一个类别 c
 - 给定类别 c ，依据概率 $P(t_k|c)$ 独立地生成每一个单词 t_k
- 为了给文档进行分类，“reengineering”上述过程，寻找能够有最大生成概率的 c

假设二：位置无关假设

- $P(X_{k_1} = t|c) = P(X_{k_2} = t|c)$
 - 假设对于属于类别 “China” 的一个文档d，其第1个位置出现Beijing和最后一个位置出现Beijing，所产生的效果是一样的
- 词袋假设(Bag of words, BOW)
 - 假设1：不同单词之间独立
 - 假设2：与位置无关



上述独立性假设是否合理？

- 条件独立： $P(\langle t_1, \dots, t_{n_d} \rangle | c) = \prod_{1 \leq k \leq n_d} P(X_k = t_k | c)$
- 位置独立： $\hat{P}(X_{k_1} = t | c) = \hat{P}(X_{k_2} = t | c)$
- 在实际的文本写作中，上述假设并不成立
 - 请举出一个例子，说明假设一（条件独立假设）并不成立。
 - 请举出一个例子，说明假设二（位置无关假设）并不成立。
- 问题：既然独立性假设被违背，为何朴素贝叶斯仍然有效？

朴素贝叶斯为何仍然有效？

- 朴素贝叶斯在条件独立假设被**严重违背**的情况下，任然有效！
- 举例：

	c_1	c_2	class selected
true probability $P(c d)$	0.6	0.4	c_1
$\hat{P}(c) \prod_{1 \leq k \leq n_d} \hat{P}(t_k c)$	0.00099	0.00001	
NB estimate $\hat{P}(c d)$	0.99	0.01	c_1

- 违背条件独立性假设，会重复利用单词的分类贡献，导致对概率的估计要么不足(underestimation, 0.01)，要么太过 overestimation (0.99)
- 但是：分类问题的目标为正确预测类别，而不是精确估计概率
- 在条件独立假设被**严重违背**的情况下，朴素贝叶斯
 - **不能**够精确估计文档属于某一个类别的概率
 - 但是通常还是能够“猜”对类别（违背独立性假设并未严重到颠覆类别的对比）

并不“朴素”的朴素贝叶斯

- 赢得KDD-CUP 1997年冠军
- 相对于大部分传统的机器学习模型
 - 对无关的特征比较鲁棒
 - 对topic drift (比如：类别的定义随着时间而变化) 鲁棒
- 相对决策树
 - 如果一些特征同样重要，则朴素贝叶斯表现更好
- 如果独立性假设成立，会有惊艳的表现
- 是文本分类中一类重要的模型，经常被用作基线模型
 - 简单：没有超参数
 - 时间：速度快、效率高
 - 空间：需要存储空间少



提纲



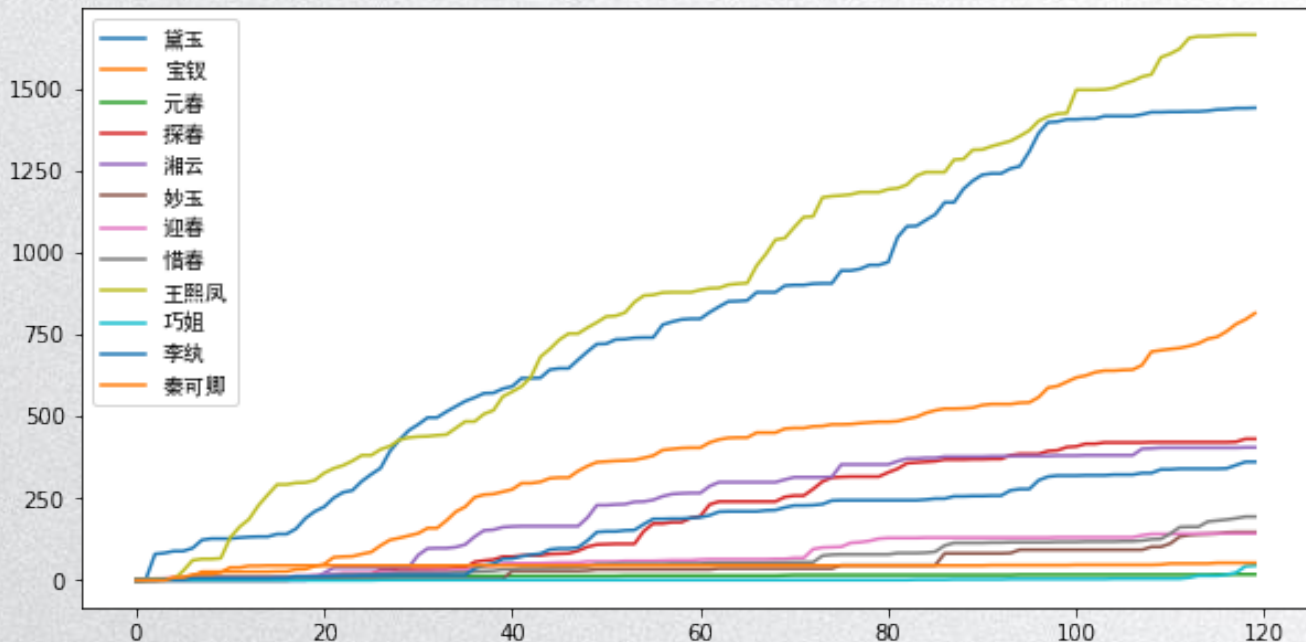
文本分类

- 1. 文本分类问题背景
- 2. 文本的向量化表示
- 3. 文本分类方法：朴素贝叶斯 (Naïve Bayes)
- 4. 文本分析实践



案例分析1：红楼梦文本分析

- 给定你关心的一组人物，统计他（她）们在不同章节出现的频率



- 课堂练习：给定你关心的一组人物，统计他们两两共同出现的章节数



案例分析2：虚假新闻数据集

- 我们生活在一个富媒体的时代，众多繁杂的信息以各种各样的形式涌入入到我们日常生活的学习工作之中，可是却同时带来了一个重要的问题——信息失真。
- fake_news.csv是一个虚假新闻数据集，包含两列，第一列列为“id”，第二部分为“context”，是新闻标题。
- 请完成任务：
 - 构建所有词的倒排索引
 - 给出以下查询，使用倒排索引找出包含所有关键词的新闻标题
 - Q1: 中国 男人
 - Q2: 演员 甩大牌



自然语言分析工具

- **NLTK**：处理自然语言数据的Python程序和平台
- **Gensim**：可扩展的统计语义、分析纯文本文档的语义结构、检索语义相似的文档
- **TextBlob**：用于处理文本数据的Python库
- **MBSP for Python**：基于TiMBL和MBT内存的学习应用程序
- **xTAS**：基于Celery的分布式文本分析套件
- **Pattern**：Python编程语言的Web挖掘模块

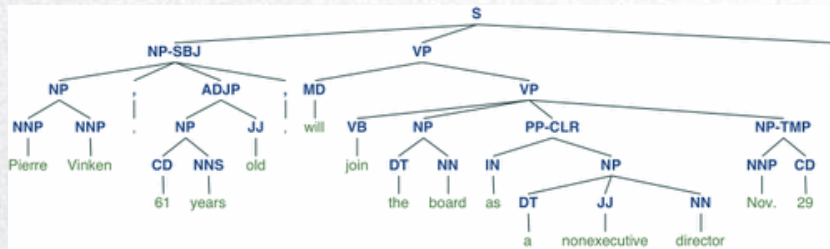


Natural Language Toolkit (NLTK)

- <http://www.nltk.org/index.html>
<https://github.com/nltk/nltk>
- 安装
 - NLTK requires Python versions 2.7, 3.4, 3.5, 3.6, or 3.7
 - 1.安装NLTK：执行`sudo pip install -U nltk`
 - 2.安装Numpy（可选）：执行`sudo pip install -U numpy`
 - 3.安装测试：执行`pyhon`，进入`python`环境后再执行`import nltk`
- 数据下载（可选）
 - NLTK附带了许多语料库，toy grammar以及训练模型等。完整的列表发布在：
http://nltk.org/nltk_data/
 - 要安装这些数据，首先安装NLTK，再利用NLTK的数据下载器进行下载

Naïve Bayes仅仅是NLTK很小的一部分

- 自然语言处理
 - Tokenization and POS tagging
 - Identify named entities
 - parse tree
 - Stemming
 - 情感分类
 - 语言模型
- 机器翻译
- 分类
 - ConditionalExponentialClassifier
 - **DecisionTreeClassifier**
 - MaxentClassifier
 - **NaiveBayesClassifier**
- 聚类
 - EM
 - kMeans



监督学习模型训练使用流程



训练样本

人工标注



Y

N

Y

N



未观测样本



特征
抽取

分类预测
算法

Y/N?

数据准备

特征抽取

模型训练

分类测试/应用



使用NLTK朴素贝叶斯进行分类实验



```
from nltk.classify import NaiveBayesClassifier
train = [
    (dict(a=1,b=1,c=1), 'y'),
    (dict(a=1,b=1,c=1), 'x'),
    (dict(a=1,b=1,c=0), 'y'),
    (dict(a=0,b=1,c=1), 'x'),
    (dict(a=0,b=1,c=1), 'y'),
    (dict(a=0,b=0,c=1), 'y'),
    (dict(a=0,b=1,c=0), 'x'),
    (dict(a=0,b=0,c=0), 'x'),
    (dict(a=0,b=1,c=1), 'y'),
]
test = [
    (dict(a=1,b=0,c=1)), # unseen
    (dict(a=1,b=0,c=0)), # unseen
    (dict(a=0,b=1,c=1)), # seen 3 times, labels=y,y,x
    (dict(a=0,b=1,c=0)), # seen 1 time, label=x
]

#train
classifier = NaiveBayesClassifier.train(train)

#test
labels = classifier.classify_many(test)
print(labels)

#show probabilities
probs = classifier.prob_classify_many(test)
for pdist in probs:
    print('%0.4f %0.4f' % (pdist.prob('x'), pdist.prob('y')))

#how these features work
classifier.show_most_informative_features()
```

['y', 'x', 'y', 'x']

0.3203 0.6797

0.5857 0.4143

0.3792 0.6208

0.6470 0.3530

Most Informative Features

c = 0

c = 1

a = 1

b = 0

a = 0

b = 1

x : y

y : x

y : x

x : y

x : y

y : x

=

=

=

=

=

=

2.0 : 1.0

1.5 : 1.0

1.4 : 1.0

1.2 : 1.0

1.2 : 1.0

1.1 : 1.0



使用说明

- 训练和测试数据集合分别都被表示为list
- List中每一个元素为一个pair：第一个为特征，第二个为分类标签
- 每一个特征（一个文档）都被表示成一个dict
- 训练模型：`classifier = NaiveBayesClassifier.train(train)`
- 测试模型：`labels = classifier.classify_many(test)`
- 得到分类的概率：`probs = classifier.prob_classify_many(test)`
- 查看每一维特征的作用：
`classifier.show_most_informative_features()`



文本二值分类

```
train_corpus = [('The team dominated the game', True),
                ('The game was intense', True),
                ('The ball went off the court', True),
                ('They had the ball for the whole game', True),
                ('The President did not comment', False),
                ('The show is over', False),
                ]

#build features
v = voc(train_corpus)

#train
train_feature_label = feature(train_corpus, v)
NBC = NaiveBayesClassifier.train(train_feature_label)

#test
test_corpus = [('I lost the keys', False),
               ('The goalkeeper caught the ball', True),
               ('The other team controlled the ball', True),
               ('Sara has two kids', False),
               ('this is a book', True),
               ]

test_feature = []
test_labels = []
for (ftr, label) in feature(test_corpus, v):
    test_feature.append(ftr)
    test_labels.append(label)

pred_labels = NBC.classify_many(test_feature)
print(test_labels)
print(pred_labels)

print('Precision = %.4f, Recall = %.4f, F-score = %.4f, Accuracy = %.4f' %
      classify_eval(test_labels, pred_labels))

# show probabilities
probs = NBC.prob_classify_many(test_feature)
for pdist in probs:
    print('%.4f %.4f' % (pdist.prob(True), pdist.prob(False)))
```

```
from nltk.classify import NaiveBayesClassifier
```

```
def voc(data):
    voc = {}
    for (sentence, val) in data:
        words = sentence.lower().split()
        for w in words:
            voc[w] = True
    return voc
```

```
def feature(data, v):
    ftr = []
    for (sentence, label) in data:
        f = dict((w, 0) for w in dict.keys(v))
        words = sentence.lower().split()
        for w in words:
            f[w] = 1
        ftr.append((f, label))
    return ftr
```

特征（字典）构建 和 特征抽取



结果评价



```
def classify_eval(truth, pred):
    idx = 0
    (TP, FP, TN, FN) = (0, 0, 0, 0)
    for truth_label in truth:
        pred_label = pred[idx]
        if (truth_label == True and pred_label == True):
            TP = TP + 1
        elif (truth_label == False and pred_label == False):
            TN = TN + 1
        elif (truth_label == True and pred_label == False):
            FN = FN + 1
        elif (truth_label == False and pred_label == True):
            FP = FP + 1
        idx = idx + 1
    P = 0 if TP == 0 else TP / (TP + FP)
    R = 0 if TP == 0 else TP / (TP + FN)
    F = 0 if (P == 0 or R == 0) else 2 * P * R / (P + R)
    Acc = 0 if (TP + TN == 0) else (TP + TN) / (TP + TN + FP + FN)
    return (P, R, F, Acc)
```

```
[False, True, True, False, True]
[True, True, True, True, False]
Precision = 0.5000, Recall = 0.6667, F-score = 0.5714, Accuracy = 0.4000
0.7776 0.2224
0.9459 0.0541
0.9740 0.0260
0.6602 0.3398
0.1775 0.8225
Most Informative Features
          game = 0          False : True   =      2.8 : 1.0
        comment = 0        True : False   =      1.8 : 1.0
           not = 0        True : False   =      1.8 : 1.0
          show = 0        True : False   =      1.8 : 1.0
          over = 0        True : False   =      1.8 : 1.0
    president = 0        True : False   =      1.8 : 1.0
           did = 0        True : False   =      1.8 : 1.0
           is = 0         True : False   =      1.8 : 1.0
          ball = 0        False : True    =      1.7 : 1.0
         whole = 0        False : True    =      1.2 : 1.0
```

哪些单词对分类最有意义？

NBC.show_most_informative_features(10)

```
[False, True, True, False, True]
[True, True, True, True, False]
Precision = 0.5000, Recall = 0.6667, F-score = 0.5714, Accuracy = 0.4000
0.7776 0.2224
0.9459 0.0541
0.9740 0.0260
0.6602 0.3398
0.1775 0.8225
Most Informative Features
      game = 0          False : True   =      2.8 : 1.0
    comment = 0        True  : False  =      1.8 : 1.0
          not = 0        True  : False  =      1.8 : 1.0
          show = 0       True  : False  =      1.8 : 1.0
          over = 0       True  : False  =      1.8 : 1.0
    president = 0       True  : False  =      1.8 : 1.0
          did = 0       True  : False  =      1.8 : 1.0
          is = 0        True  : False  =      1.8 : 1.0
          ball = 0      False : True    =      1.7 : 1.0
          whole = 0     False : True    =      1.2 : 1.0
```



使用SVM进行文本分类

from sklearn.svm import LinearSVC

```
SVMC = nltk.classify.SklearnClassifier(LinearSVC())
SVMC.train(train_feature_label)
pred_labels_SVM = []
for f in test_feature:
    pred_labels_SVM.append(SVMC.classify(f))

print(pred_labels_SVM)
print('Precision = %.4f, Recall = %.4f, F-score = %.4f, Accuracy = %.4f'%
      classify_eval(test_labels, pred_labels_SVM))
```

利用sklearn进行情感分析

- 真实数据集dianping.csv – 来自大众点评

	A	B
1	comment	star
2	口味：不知道是我口高了，还是这家真不怎么样。??我感觉口味确实很一般很一般。上菜相当快，我敢说菜都是提前做好的	2
3	菜品丰富质量好，服务也不错！很喜欢！	4
4	说真的，不晓得有人排队的理由，香精香精香精香精，拜拜！	2
5	菜量实惠，上菜还算比较快，疙瘩汤喝出了秋日的暖意，烧茄子吃出了大阪烧的味道，想吃土豆片也是口感绵软停不下来，	5
6	先说我算是娜娜家风荷园开业就一直在这里吃??每次出去回来总想吃一回??有时觉得外面的西式简餐总是不如天津的某些家	4
7	菜很好吃??环境也不错????值得推荐!!!!!!!!!!!!!!!!!!!!	5
8	很喜欢的餐厅，东北风味儿，以后还会再去！	4
9	口味属于家常菜一类，价格有点小贵，逛街的时候吃吃还可以。	4
10	最近太忙了，估计要暂时告别大众点评一段时间了。这条应该是这个月的最后一条点评了，全五星好评。在天津连续下雨的	5
11	我去了一次感觉喜欢上了这，无论环境还有菜品在自助餐里已经是很不错了	5
12	10点半点的餐，12点还没下单，这速度也是醉了。味道还可以，德国香肠没有太好吃，焗土豆和柠檬鸡还是推荐的。	2
13	两个人吃没点太多，要了一份手抓饼当主食，没仔细看菜单，以为手抓饼是咸口的了??结果太甜了，油还有点大。当时特别	4
14	很一般????地点好真的很重要????????不是很推荐	2
15	我们两个人去的，等个100多个号吧，还行因为不着急，就特地为了去拔草，点了一直烤鸭，味道不错，就是上的特别慢，	2
16	哎，冲着推荐来的，后悔死了，上菜巨慢不说，服务态度还超级差，东西咸的要死，而且普遍的贵，谁来谁犯二，这辈子不	1
17	超级差评????上菜慢??服务态度超级差、先把汤上来喝饱了??完事菜就上了一个，找服务员催半天也上不来最后结账态度超	1
18	很好吃，中餐有西餐味。用餐人很多，我们排队了15分钟呢，爆	5
19	平日晚餐去的时间不算晚人也依旧那么火爆??等了一个小时??安排了一张超小的桌子??只适合下午茶用??很无语??点了奶酪	5
20	好吃好吃好吃，就是人不少！烤肉速度人多时候稍有些慢！	5
21	总体还不错??要的都是推荐菜??手撕饼有点腻	5
22	没有菜单??用微信点菜??还是先付款??什么都没吃着就把钱付了??忙活点餐付款就十几二十分钟??还真得不饿的时候来吃饭	1
23	上周末去的娜娜家，一直没来得及点评，今天趁着工作不忙赶紧来补一个??作为一个爱吃甜食的女生，真的是对这种有很多	5
24	服务员跟**似的，摇摇手，没位置，话都不说一句。	1
25	自从这家出了就一直想去，终于赶上闺蜜也有时间的日子早早的去了。不到五点半到那就已经排到10几号了，差不多等了半	5
26	东西什么的都挺好，味道也不错，就是人多点，环境很好	5
27	早就听闻卤肉饭，可是不太喜欢吃，是甜口的，米饭太硬太硬了应该是隔夜的，肉也不是很烂！！烟花意面挺辣的，就是意	2

读取数据

- 如何从CSV文件中读取数据？
 - 使用pandas !

```
In [2]: data = pd.read_csv('datasets/dianping.csv')  
data.head()
```

Out[2]:

		comment	star
0	口味：不知道是我口高了，还是这家真不怎么样。??我感觉口味确实很一般很一般。上菜相当快，我敢...		2
1		菜品丰富质量好，服务也不错！很喜欢！	4
2		说真的，不晓得有人排队的理由，香精香精香精香精，拜拜！	2
3	菜量实惠，上菜还算比较快，疙瘩汤喝出了秋日的暖意，烧茄子吃出了大阪烧的味道，想吃土豆片也是口...		5
4	先说我算是娜娜家风荷园开业就一直在这里吃??每次出去回来总想吃一回??有时觉得外面的西式简餐...		4

数据预处理

- 根据star属性设置sentiment属性
 - 三星以上 → positive ; 三星及以下 → negative

```
In [3]: def make_label(star):  
        if star > 3:  
            return 1  
        else:  
            return 0  
  
data['sentiment'] = data.star.apply(make_label)  
data = data[['comment' , 'sentiment']]  
data.head()
```

Out[3]:

	comment	sentiment
0	口味：不知道是我口高了，还是这家真不怎么样。??我感觉口味确实很一般很一般。上菜相当快，我敢...	0
1	菜品丰富质量好，服务也不错！很喜欢！	1
2	说真的，不晓得有人排队的理由，香精香精香精香精，拜拜！	0
3	菜量实惠，上菜还算比较快，疙瘩汤喝出了秋日的暖意，烧茄子吃出了大阪烧的味道，想吃土豆片也是口...	1
4	先说我算是娜娜家风荷园开业就一直在这里吃??每次出去回来总想吃一回??有时觉得外面的西式简餐...	1

小白版情感分析：snownlp库

```
In [4]: from snownlp import SnowNLP

text1 = '这个东西不错'
text2 = '这个东西很垃圾'

s1 = SnowNLP(text1)
s2 = SnowNLP(text2)

print(s1.sentiments,s2.sentiments)

def snow_result(comemnt):
    s = SnowNLP(comemnt)
    if s.sentiments >= 0.6:
        return 1
    else:
        return 0

data['snlp_result'] = data.comment.apply(snow_result)
data.head()
```

```
0.8623218777387431 0.21406279508712744
```

Out[4]:

	comment	sentiment	snlp_result
0	口味: 不知道是我口高了, 还是这家真不怎么样。??我感觉口味确实很一般很一般。上菜相当快, 我敢...	0	0
1	菜品丰富质量好, 服务也不错! 很喜欢!	1	1
2	说真的, 不晓得有人排队的理由, 香精香精香精香精, 拜拜!	0	1
3	菜量实惠, 上菜还算比较快, 疙瘩汤喝出了秋日的暖意, 烧茄子吃出了大阪烧的味道, 想吃土豆片也是口...	1	0
4	先说我算是娜娜家风荷园开业就一直在这里吃??每次出去回来总想吃一回??有时觉得外面的西式简餐...	1	0

小白版情感分析：snownlp库

Out[4]:

	comment	sentiment	snlp_result
0	口味：不知道是我口高了，还是这家真不怎么样。??我感觉口味确实很一般很一般。上菜相当快，我敢...	0	0
1	菜品丰富质量好，服务也不错！很喜欢！	1	1
2	说真的，不晓得有人排队的理由，香精香精香精香精，拜拜！	0	1
3	菜量实惠，上菜还算比较快，疙瘩汤喝出了秋日的暖意，烧茄子吃出了大阪烧的味道，想吃土豆片也是口...	1	0
4	先说我算是娜娜家风荷园开业就一直在这里吃??每次出去回来总想吃一回??有时觉得外面的西式简餐...	1	0

- 使用snownlp库的优缺点

- 优点：简单方便
- 缺点：准确率低

```
from snownlp import SnowNLP
text1 = '这个东西不错'
text2 = '这个东西很垃圾'
s1 = SnowNLP(text1)
s2 = SnowNLP(text2)
print(s1.sentiments,s2.sentiments)
```

分词：准备词袋模型

- 使用jieba分词进行简单分词

```
def simple_word_cut (texts):
    return ' '.join(jieba.cut(texts, cut_all=False))

data['simple_cut_comment'] = data.comment.apply(simple_word_cut)
data.head()
```

```
Building prefix dict from the default dictionary ...
Loading model from cache /var/folders/8d/62lh93xj5lv9v1r3f8qg6m6r0000gn/T/jieba.cache
Loading model cost 0.975 seconds.
Prefix dict has been built successfully.
```

	comment	sentiment	simple_cut_comment
0	口味：不知道是我口高了，还是这家真不怎么样。??我感觉口味确实很一般很一般。上菜相当快，我敢...	0	口味： 不知道 是 我口 高 了 ， 还 是 这 家 真 不 怎 么 样 。 ?? 我 感 觉 口...
1	菜品丰富质量好，服务也不错！很喜欢！	1	菜品 丰富 质量 好 ， 服 务 也 不 错 ！ 很 喜 欢 ！
2	说真的，不晓得有人排队的理由，香精香精香精香精，拜拜！	0	说 真 的 ， 不 晓 得 有 人 排 队 的 理 由 ， 香 精 香 精 香 精 香 精 ， 拜 拜 ！
3	菜量实惠，上菜还算比较快，疙瘩汤喝出了秋日的暖意，烧茄子吃出了大阪烧的味道，想吃土豆片也是口...	1	菜 量 实 惠 ， 上 菜 还 算 比 较 快 ， 疙 瘩 汤 喝 出 了 秋 日 的 暖 意 ， 烧 茄 子 吃 ...
4	先说我算是娜娜家风荷园开业就一直在这里吃??每次出去回来总想吃一回??有时觉得外面的西式简餐...	1	先 说 我 算 是 娜 娜 家 风 荷 园 开 业 就 一 直 在 这 里 吃 ?? 每 次 出 去 回 来 总 ...

去除停用词 (stopwords)

In [9]:

```
def word_cut (texts):
    words_list = []
    word_generator = jieba.cut(texts, cut_all=False) # 返回的是一个迭代器
    with open('hit_stopwords.txt') as f:
        str_text = f.read()
        for word in word_generator:
            if word.strip() not in str_text:
                words_list.append(word)
            #print ('1')
    return ' '.join(words_list) # 注意是空格

data['cut_comment'] = data.comment.apply(word_cut)

data.head()
```

Out[9]:

	comment	sentiment	cut_comment
0	口味：不知道是我口高了，还是这家真不怎么样。??我感觉口味确实很一般很一般。上菜相当快，我敢...	0	口味 知道 我口 高 这家 不怎么样 感觉 口味 确实 很 很 上菜 相当 快 我敢 菜 都...
1	菜品丰富质量好，服务也不错！很喜欢！	1	菜品 丰富 质量 服务 不错 很 喜欢
2	说真的，不晓得有人排队的理由，香精香精香精香精，拜拜！	0	说真的 晓得 有人 排队 理由 香精 香精 香精 香精 拜拜
3	菜量实惠，上菜还算比较快，疙瘩汤喝出了秋日的暖意，烧茄子吃出了大阪烧的味道，想吃土豆片也是口...	1	菜量 实惠 上菜 算 比较 快 疙瘩汤 喝出 秋日 暖意 烧茄子 吃 出 大阪 烧 味道 想...
4	先说我算是娜娜家风荷园开业就一直在这里吃??每次出去回来总想吃一回??有时觉得外面的西式简餐...	1	先说 算是 娜娜 家风 荷园 开业 吃 每次 出去 回来 总想 吃 一回 有时 觉得 外面 ...

提取特征

```

: X = data['cut_comment']
  y = data['sentiment']

  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

: from sklearn.feature_extraction.text import TfidfVectorizer

  vect = TfidfVectorizer(max_df = 0.8,
                        min_df = 3,
                        token_pattern=u'(?u)\\b[^\d\\W]\\w+\\b'
                        )

  features = pd.DataFrame(vect.fit_transform(X_train).toarray(), columns=vect.get_feature_nam
  features.head()

```

```

:

```

	app	ipad	ok	ps	wifi	一下	一个个	一个半	一个多	一人	...	麻将	麻烦	麻辣	麻酱	黄瓜	黄盖	黄色	黑椒	默默	齐全
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 1910 columns

使用朴素贝叶斯分类器

```
In [13]: from sklearn.naive_bayes import MultinomialNB
```

```
nb = MultinomialNB()
```

```
X_train_vect = vect.fit_transform(X_train)
```

```
nb.fit(X_train_vect, y_train)
```

```
train_accuracy = nb.score(X_train_vect, y_train)
```

```
print (train_accuracy)
```

```
0.914375
```

```
In [14]: X_test_vect = vect.transform(X_test)
test_accuracy = nb.score(X_test_vect, y_test)
```

```
y_predict = nb.predict(X_test_vect)
```

```
print('测试准确率', test_accuracy)
```

```
from sklearn.metrics import classification_report
```

```
print("测试集上其他指标: \n", classification_report(y_test, y_predict))
```

```
测试准确率 0.82
```

```
测试集上其他指标:
```

	precision	recall	f1-score	support
0	0.85	0.77	0.81	197
1	0.79	0.87	0.83	203
micro avg	0.82	0.82	0.82	400
macro avg	0.82	0.82	0.82	400
weighted avg	0.82	0.82	0.82	400



谢谢！